JAVASCRIPT BASICS

Beginner-friendly student notes • Concepts, examples and practice

Topics covered
Code execution • Variables • Data types • Type coercion • Boolean logic • Strings • Arrays • Conditions • Loops • Error handling • Objects • Operators

These notes are designed for beginners. Read the explanation, type each example in VS Code, run it, and observe the output.

Practice tip: Predict the output before running the code.

1. How JavaScript code executes

JavaScript is a programming language used to make web pages interactive. A **JavaScript engine** is the program that reads and executes JavaScript. For example, Google Chrome uses the V8 engine.

Parsing means the engine reads the code and checks its structure and syntax. **Execution** means carrying out the instructions.

```
console.log("Start");
console.log("I am learning JavaScript");
console.log("End");
```

Output: Start → I am learning JavaScript → End

A JavaScript engine processes code and executes it. In a browser, the engine works with other browser features to handle tasks such as timers and page interaction.

A first look at a timer

```
console.log("Start");

setTimeout(function() {
    console.log("Timer");
}, 3000);

console.log("End");
```

Output order: Start, End, Timer. The timer callback runs after about 3 seconds. This is a first look at asynchronous behaviour.

2. Variables

A **variable** is a named place used to store a value.

```
let name = "Rahul";
let age = 22;
const city = "Bengaluru";

console.log(name);
console.log(age);
console.log(city);
```

Keyword	Beginner explanation
var	Older way to declare a variable.
let	Use when you may assign a new value later.
const	Use when you will not reassign the variable.

```
let marks = 70;
marks = 85;
console.log(marks); // 85

const country = "India";
// country = "Japan"; // TypeError: cannot reassign a const
```

Remember: A const variable cannot be reassigned. The contents of an object or array declared with const can still be changed.

3. Data types

A data type tells us what kind of value is stored.

Type	Example
String	"Hello"

Type	Example
Number	25 or 99.5
Boolean	true or false
Undefined	let city;
Null	let address = null;
Object	{ name: "Rahul" }

```
let studentName = "Anjali";
let age = 21;
let isStudent = true;
let city;
let address = null;

console.log(typeof studentName); // string
console.log(typeof age);         // number
console.log(typeof isStudent);   // boolean
console.log(typeof city);        // undefined
console.log(typeof address);     // object (JavaScript quirk)
```

Quick check: "100" is a string because it is inside quotes. 100 is a number.

4. Variable mutation and type coercion

Variable mutation

Mutation in this beginner example means changing the value assigned to a variable.

```
let score = 70;
score = 85;
console.log(score); // 85
```

Type coercion

Type coercion is JavaScript converting a value from one data type to another, sometimes automatically.

```
console.log("10" + 5); // "105" (string concatenation)
console.log("10" - 5); // 5 (string converted to number)
console.log(10 + 5);   // 15
```

The + operator can join strings. The - operator performs numeric subtraction, so JavaScript converts the numeric string.

5. Boolean values and logic

A Boolean has only two values: **true** or **false**. Comparisons produce Boolean results.

```
let age = 22;
console.log(age > 18);  // true
console.log(age < 18);  // false
console.log(age === 22); // true
```

Operator	Meaning	Example idea
&&	AND: both conditions must be true	age >= 18 && hasID
||	OR: at least one condition must be true	isWeekend || isHoliday
!	NOT: reverses true/false	!isLoggedIn

```
let age = 22;
let hasID = true;
console.log(age >= 18 && hasID); // true
```

6. String properties and methods

A **string** is text made up of characters. String methods help us inspect or work with text. Most string methods return a new value; they do not change the original string.

```
let message = "Hello JavaScript";
console.log(message.length); // 16
```

Method / property	Purpose	Example
length	Counts characters	message.length
toUpperCase()	Converts to uppercase	message.toUpperCase()
toLowerCase()	Converts to lowercase	message.toLowerCase()
trim()	Removes spaces from both ends	" Hi ".trim()
charAt(index)	Returns character at position	message.charAt(0)
charCodeAt(index)	Returns UTF-16 code unit number	"A".charCodeAt(0) → 65
concat()	Joins strings	"Hello".concat(" ", "Sam")
repeat(n)	Repeats a string n times	"Hi ".repeat(3)
includes()	Checks whether text exists	message.includes("Java")
match()	Finds matches using a pattern	message.match(/Java/)
replace()	Replaces the first matching text	message.replace("Java", "Type")
search()	Returns position of a match, or -1	message.search("Java")
slice(start,end)	Extracts part of a string	message.slice(0, 5)

Simple examples

```
let name = "rahul";

console.log(name.toUpperCase()); // RAHUL
console.log(name.charAt(1));     // a
console.log("A".charCodeAt(0));  // 65
console.log("Hello ".repeat(2)); // Hello Hello
console.log("Hello World".includes("World")); // true
console.log("Hello World".replace("World", "JS")); // Hello JS
console.log("Hello World".search("World")); // 6
console.log("Hello World".slice(0, 5)); // Hello
```

Note: charCodeAt() returns a UTF-16 code unit (for common basic characters this matches the familiar Unicode number). match() uses a pattern; for example /Java/ searches for the text Java.

```
let sentence = "I love JavaScript";
console.log(sentence.match(/JavaScript/)); // matching result
console.log(sentence.match(/Python/));     // null
```

7. Arrays and array operations

An **array** stores multiple values in one variable. Array positions are called indexes, and the first index is 0.

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]); // Apple
console.log(fruits[1]); // Banana
console.log(fruits.length); // 3
```

Method	Purpose
push(item)	Adds item at the end

Method	Purpose
pop()	Removes the last item
unshift(item)	Adds item at the beginning
shift()	Removes the first item
splice(start, count, ...items)	Adds, removes or replaces items; changes original array
slice(start, end)	Returns a selected portion; does not change original array
concat(array)	Joins arrays and returns a new array
sort()	Sorts array items; changes original array
forEach()	Runs a function once for each item

Adding and removing items

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.push("Orange");    // add at end
fruits.pop();             // remove last item
fruits.unshift("Grapes"); // add at beginning
fruits.shift();           // remove first item

console.log(fruits);
```

splice() – add or remove

```
let fruits = ["Apple", "Banana", "Mango"];
fruits.splice(1, 1); // from index 1, remove 1 item
console.log(fruits); // ["Apple", "Mango"]

fruits.splice(1, 0, "Orange"); // add at index 1, remove 0
console.log(fruits); // ["Apple", "Orange", "Mango"]
```

slice() – take a portion

```
let fruits = ["Apple", "Banana", "Mango", "Orange"];
let selected = fruits.slice(1, 3);
console.log(selected); // ["Banana", "Mango"]
console.log(fruits);   // original array is unchanged
```

concat() – join arrays

```
let groupA = ["Rahul", "Arun"];
let groupB = ["Priya", "Anjali"];
let students = groupA.concat(groupB);
console.log(students);
```

sort() – arrange items

```
let fruits = ["Mango", "Apple", "Banana"];
fruits.sort();
console.log(fruits); // ["Apple", "Banana", "Mango"]

let numbers = [40, 10, 100, 5];
numbers.sort(function(a, b) {
    return a - b; // ascending numeric order
});
console.log(numbers); // [5, 10, 40, 100]
```

Without a comparison function, sort() arranges values as strings. Use function(a, b) { return a - b; } for ascending numeric order.

forEach() – visit each item

```
let fruits = ["Apple", "Banana", "Mango"];

fruits.forEach(function(fruit) {
    console.log(fruit);
```

```
});
```

Remember: splice() changes the original array. slice() returns a portion without changing the original array.

8. Statements and decision-making

A **statement** is an instruction that JavaScript can execute.

if / else

Use if/else when a program needs to make a decision.

```
let age = 20;

if (age >= 18) {
    console.log("Eligible to vote");
} else {
    console.log("Not eligible to vote");
}
```

else if

Use else if when there are multiple conditions to check.

```
let marks = 75;

if (marks >= 90) {
    console.log("Grade A");
} else if (marks >= 60) {
    console.log("Grade B");
} else if (marks >= 35) {
    console.log("Grade C");
} else {
    console.log("Fail");
}
```

9. Loops and iteration

A **loop** repeats a block of code. **Iteration** means going through items one by one.

for loop

```
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

Start: let i = 1 • Condition: i <= 5 • Update: i++

while loop

```
let i = 1;
while (i <= 5) {
    console.log(i);
    i++;
}
```

do...while loop

The code runs at least once before the condition is checked.

```
let i = 1;
do {
    console.log(i);
    i++;
} while (i <= 5);
```

for...of loop

A simple way to go through values in an array.

```
let fruits = ["Apple", "Banana", "Mango"];

for (let fruit of fruits) {
    console.log(fruit);
}
```

Loop	When to use
for	When you know the number of repetitions
while	Repeat while a condition is true
do...while	Run at least once, then check
for...of	Go through values in an iterable, such as an array

10. Error handling

Error handling lets a program respond to certain errors instead of allowing an error to interrupt the expected flow.

Keyword	Purpose
try	Contains code that may cause an error
catch	Runs if an error is thrown in try
finally	Runs after try/catch whether an error occurred or not
throw	Creates or throws an error

```
try {
    console.log("Start");
    console.log(unknownVariable);
} catch (error) {
    console.log("Something went wrong");
} finally {
    console.log("Program completed");
}
```

The undefined variable causes a ReferenceError. The catch block handles it, and finally runs afterward.

throw – create an error

```
let age = 15;

try {
    if (age < 18) {
        throw new Error("Age must be 18 or above");
    }
    console.log("Eligible");
} catch (error) {
    console.log(error.message);
}
```

11. JavaScript objects, properties and methods

An **object** stores related information as key-value pairs. The keys are called **properties**. A function stored in an object is called a **method**.

```
let student = {
    name: "Rahul",
    age: 22,
    course: "JavaScript"
};

console.log(student.name);    // Rahul
console.log(student["age"]);  // 22
```

Update and add properties

```
student.age = 23;                  // update a property
student.city = "Bengaluru";       // add a property
console.log(student);
```

Object method

```
let student = {
    name: "Rahul",
    introduce: function() {
        console.log("Hello, my name is " + this.name);
    }
};

student.introduce();
```

this.name refers to the name property of this student object when the method is called as student.introduce().

12. JavaScript operators

Operators are symbols that perform operations on values and variables.

Arithmetic operators

Operator	Meaning
+	Addition (or string joining)
-	Subtraction
*	Multiplication
/	Division
%	Remainder
**	Exponent / power

```
let a = 10;
let b = 5;
console.log(a + b); // 15
console.log(a - b); // 5
console.log(a * b); // 50
console.log(a / b); // 2
console.log(a % b); // 0
console.log(2 ** 3); // 8
```

Assignment operators

```
let marks = 50;
marks += 10; // same as marks = marks + 10
console.log(marks); // 60
```

Other common assignment operators: -=, *=, /=.

Comparison operators

```
let a = 10;
let b = 20;
console.log(a > b);   // false
console.log(a < b);   // true
console.log(5 == "5");  // true: loose equality
console.log(5 === "5"); // false: strict equality
```

Tip: Prefer === when comparing values because it checks both value and type.

Logical operators

&& means AND (both true); || means OR (at least one true); ! means NOT (reverses a Boolean result).

```
let age = 22;
let hasID = true;
console.log(age >= 18 && hasID); // true
```

Bitwise operators – introduction

Bitwise operators work on the binary (0 and 1) representation of numbers. For now, learn their names.

Operator	Name
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift

```
console.log(5 & 1); // 1
console.log(5 | 1); // 5
```

Ternary operator

The ternary operator is a short way to write a simple if/else expression.

```
let age = 20;
let result = age >= 18 ? "Eligible" : "Not Eligible";
console.log(result); // Eligible
```

Pattern: condition ? valueIfTrue : valueIfFalse.

13. Practice exercises

Try these exercises yourself before checking the examples from class.

1. Create variables for your name, age and course. Print them using console.log().

2. Create a variable with the value 50, update it to 75, and print the new value.

3. Check whether the number 25 is greater than 18.

4. Convert the string "javascript" to uppercase.

5. Check whether "Hello World" includes the word "World".

6. Create an array of four subjects. Add one subject, remove the last subject, and print the array.

7. Use slice() to get the second and third items from an array.

8. Use forEach() to print every item in an array.

9. Write an if/else statement to check whether a student passed (marks >= 35).

10. Use a for loop to print numbers from 1 to 10.

11. Create a student object with name and marks properties. Print the student's name.

12. Use try/catch and throw to display "Invalid age" when age is below 18.

13. Use a ternary operator to display "Adult" or "Minor" based on age.

14. Mini practical: Student result checker

This combines variables, objects, if/else, loops, error handling and the ternary operator.

```
let student = {
    name: "Anjali",
    marks: 75
};

if (student.marks >= 35) {
    console.log(student.name + " has passed");
} else {
    console.log(student.name + " has failed");
}

for (let i = 1; i <= 5; i++) {
    console.log(i);
}

try {
    if (student.marks > 100 || student.marks < 0) {
        throw new Error("Invalid marks");
    }
    console.log("Marks are valid");
} catch (error) {
    console.log(error.message);
}

let grade = student.marks >= 60 ? "Good" : "Needs improvement";
console.log(grade);
```

Practice: Change marks to 25, 45, 85 and 110. Observe how the output changes.

15. Quick revision

Question	Remember
What is a variable?	A named place to store a value.
What is a data type?	The kind of value stored.

Question	Remember
What is an array?	A collection of values in one variable.
What is an object?	Related information stored as key-value pairs.
What is a loop?	A way to repeat code.
What is type coercion?	Conversion from one data type to another.
What does try/catch do?	Helps handle errors.
What does push() do?	Adds an item at the end of an array.
What does slice() do?	Returns a selected portion without changing the original array.
What does === check?	Value and data type.

All code examples in these notes can be practised in VS Code using a .js file and Node.js. Run a file from the terminal with: node filename.js.